RECORD

Field	Details
Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	4
Student Name	R Varshini
Roll No.	A24126552115
Date	30-08-2026

1. TITLE

Develop a Dynamic Webpage Using JavaScript

2. AIM

To develop a dynamic webpage using JavaScript that displays a live-updating clock, supports a dark mode toggle, and allows tasks to be added to and removed from a task list.

3. OBJECTIVE

The objectives of this experiment are:

- To select and manipulate DOM elements using JavaScript.
- To display a real-time clock that updates automatically every second.
- To dynamically toggle a dark mode theme by adding/removing a CSS class.
- To dynamically create and append new task items to a list based on user input.
- To attach a delete button to each task that removes it from the list when clicked.
- To handle both button clicks and the Enter key for adding a task.

4. THEORY

A dynamic webpage changes its content or appearance in response to time or user interaction, without requiring a page reload. JavaScript's Date object can be used to read the current time, and window.setInterval(fn, ms) repeatedly calls a function at a fixed time interval, which is used here to refresh a clock display every 1000 milliseconds.

classList.toggle("dark-mode") adds a CSS class to an element if it is not already present, or removes it if it is, allowing a single button click to switch between light and dark themes defined through the .dark-mode CSS rule.

Dynamic content creation uses document.createElement() to build new elements such as a task's <li> and its delete <button>, which are then inserted into the page with appendChild(). Event listeners (addEventListener("click", ...)) connect these dynamically created buttons to their own remove behaviour, so each task can delete itself using li.remove().

Handling both a button click and a keypress event (checking event.key === "Enter") for the same addTask() function demonstrates that multiple types of user interaction can trigger the same piece of application logic.

5. ALGORITHM / PROCEDURE

- Create an HTML file with a header containing a clock display and a dark-mode toggle button, and a main section with a task input, Add Task button and an empty task list.

- Link an external CSS file defining the base page style and a .dark-mode override for background and text color.
- Link an external JavaScript file to the HTML page.
- Select the theme toggle button, clock element, task input, Add Task button and task list using getElementById().
- Define an updateClock() function that reads the current time and displays it in the clock element.
- Call updateClock() once immediately, then use setInterval() to call it every second.
- Attach a click event listener to the theme toggle button that toggles the 'dark-mode' class on the document body.
- Define an addTask() function that reads and trims the task input value, alerting the user if it is blank.
- Inside addTask(), create a new list item and a delete button using createElement().
- Attach a click event listener to the delete button that removes its parent list item from the page.
- Append the delete button to the list item, and the list item to the task list; then clear the input field.
- Attach addTask() to the Add Task button's click event and to the Enter key on the input field.
- Open the HTML file in a browser and verify the clock updates every second, the dark mode toggle works, and tasks can be added and deleted.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Web Page</title>
    <link rel="stylesheet" href="style2.css">
</head>
<body>

    <header>
        <!-- The live clock will update here -->
        <div id="clock">Loading time...</div>
        <button id="theme-toggle">Toggle Dark Mode</button>
    </header>

    <main>
        <h1>Dynamic Task Manager</h1>

        <div class="input-section">
            <input type="text" id="task-input" placeholder="Enter a new task...">
            <button id="add-task-btn">Add Task</button>
        </div>

        <!-- JavaScript will inject dynamic list items here -->
        <ul id="task-list"></ul>
    </main>

    <script src="script2.js"></script>
</body>
</html>
```

style2.css

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 20px;
    background-color: #ffffff;
```

```
    color: #333333;
    transition: background 0.3s, color 0.3s;
}

header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}

.input-section {
    margin-bottom: 20px;
}

input {
    padding: 8px;
    font-size: 16px;
}

button {
    padding: 8px 12px;
    font-size: 16px;
    cursor: pointer;
}

ul {
    list-style-type: none;
    padding: 0;
}

li {
    padding: 10px;
    background: #f4f4f4;
    margin-bottom: 5px;
    display: flex;
    justify-content: space-between;
    color: #333;
}

/* Dark mode overrides */
body.dark-mode {
    background-color: #1e1e1e;
    color: #ffffff;
}
```

script2.js

```
// 1. Select DOM elements
const themeToggle = document.getElementById('theme-toggle');
const clockElement = document.getElementById('clock');
const taskInput = document.getElementById('task-input');
const addTaskBtn = document.getElementById('add-task-btn');
const taskList = document.getElementById('task-list');

// 2. Dynamic Real-Time Clock
function updateClock() {
    const now = new Date();
    clockElement.textContent = now.toLocaleTimeString();
}
// Run clock immediately and refresh every 1 second
updateClock();
setInterval(updateClock, 1000);

// 3. Dynamic Styling (Dark Mode Toggle)
themeToggle.addEventListener('click', () => {
    document.body.classList.toggle('dark-mode');
});
```

```
// 4. Dynamic Content Creation (Add Tasks)
function addTask() {
    const taskText = taskInput.value.trim();

    if (taskText === '') {
        alert('Please enter a task!');
        return;
    }

    // Create a new <li> element dynamically
    const li = document.createElement('li');
    li.textContent = taskText;

    // Create a delete button for the task
    const deleteBtn = document.createElement('button');
    deleteBtn.textContent = 'X';
    deleteBtn.style.backgroundColor = '#ff4d4d';
    deleteBtn.style.color = 'white';
    deleteBtn.style.border = 'none';

    // Add click event to remove the element when clicked
    deleteBtn.addEventListener('click', () => {
        li.remove();
    });

    // Append button to the list item, and list item to the list
    li.appendChild(deleteBtn);
    taskList.appendChild(li);

    // Clear input field
    taskInput.value = '';
}

// Trigger action on button click
addTaskBtn.addEventListener('click', addTask);

// Trigger action when pressing "Enter" key inside the input
taskInput.addEventListener('keypress', (event) => {
    if (event.key === 'Enter') {
        addTask();
    }
});
```

7. CODE EXPLANATION

Code	Explanation
document.getElementById()	Selects the clock, theme toggle button, task input, Add Task button and task list elements.
new Date() / toLocaleTimeString()	Gets the current date and time and formats it as a readable time string for the clock.
setInterval(updateClock, 1000)	Calls updateClock() repeatedly every 1000 milliseconds so the clock stays current.
classList.toggle("dark-mode")	Adds the dark-mode class if absent or removes it if present, switching the page theme.
document.createElement()	Dynamically creates the new task element and its delete <button>.
li.remove()	Removes a task's list item from the DOM when its delete button is clicked.
appendChild()	Inserts the delete button into the list item, and the list item into the task list.
addEventListener("click", fn)	Registers click handlers for the theme toggle, Add Task button and each task's delete button.
addEventListener("keypress", fn)	Allows a task to be added by pressing Enter inside the input field.
taskInput.value.trim()	Reads the entered task text and removes leading/trailing whitespace before validating it.

8. TEST CASES AND EXECUTION DATA

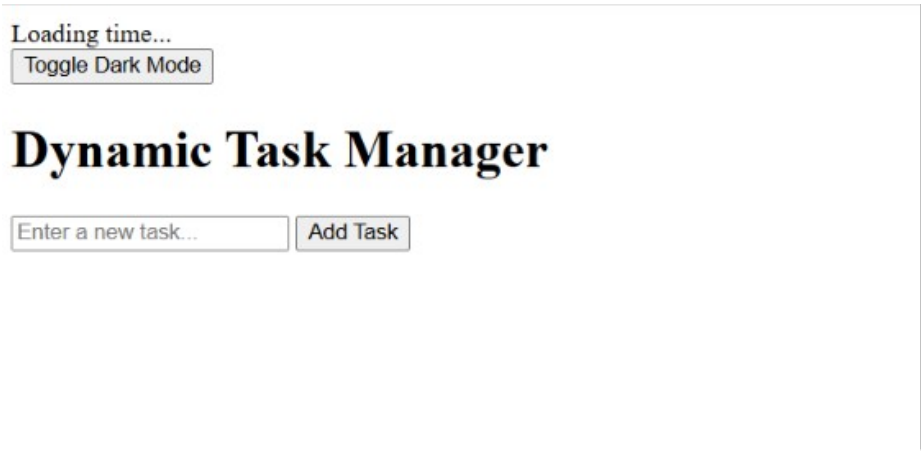
Each dynamic feature was tested by interacting with the application in the browser.

Test Case	Action	Expected Result	Status
TC-01	Page load	Clock starts showing the current time and updates every second	Pass
TC-02	Click Toggle Dark Mode	Page background and text colors switch to dark theme	Pass
TC-03	Click Toggle Dark Mode again	Page reverts to the light theme	Pass
TC-04	Add a task with text entered	New task appears in the list with a delete (X) button; input is cleared	Pass
TC-05	Click Add Task with empty input	Alert is shown; no task is added	Pass
TC-06	Press Enter after typing a task	Task is added, same as clicking Add Task	Pass
TC-07	Click the delete (X) button on a task	Task is removed from the list	Pass

9. OUTPUT

Output 1: Dynamic Task Manager Page

The page displays the live clock and Toggle Dark Mode button in the header, followed by the "Dynamic Task Manager" heading, the task input box and Add Task button.



10. RESULT

Thus, a dynamic webpage was successfully developed using JavaScript, featuring a live-updating clock, a dark mode toggle implemented through dynamic class switching, and a task manager that allows tasks to be added and removed from the DOM in real time. The application was verified using multiple test cases.